

Hotel Booking Cancellation Prediction Project Documentation

[Hotel Booking Cancellation Prediction — Project Documentation](#)

- [1. Overview](#)
- [2. Dataset](#)
 - [2.1 Feature Dictionary](#)
- [3. Environment & Dependencies](#)
- [4. Workflow](#)
 - [4.1 Data Loading](#)
 - [4.2 Data Cleaning](#)
 - [4.3 Exploratory Data Analysis \(EDA\)](#)
 - [4.4 Feature Selection \(Leakage / Low-Value Column Removal\)](#)
 - [4.5 Encoding](#)
 - [4.6 Train/Test Split \(Baseline Models\)](#)
- [5. Baseline Model Comparison](#)
- [6. Production Pipeline \(Deployment-Ready Model\)](#)
- [7. Project Structure Summary](#)
- [8. Notes](#)

Hotel Booking Cancellation Prediction — Project Documentation

1. Overview

This project builds a machine learning system that predicts whether a hotel booking will be **canceled** or **not canceled**, using the *Hotel Booking Demand* dataset (Kaggle: `jessemostipak/hotel-booking-demand`). The notebook covers the full workflow: data loading and cleaning, exploratory data analysis (EDA), feature encoding, baseline comparison across 12 classification algorithms, and a final production-ready pipeline (XGBoost, tuned via `RandomizedSearchCV`) that is serialized for deployment (e.g., behind a Gradio app).

Target variable: `is_canceled` (0 = Not Canceled, 1 = Canceled)

2. Dataset

- **Source:** Kaggle dataset `jessemostipak/hotel-booking-demand`, downloaded programmatically via `kagglehub.dataset_download(...)`.
- **Raw shape:** 31 features (columns) before cleaning.

2.1 Feature Dictionary

Feature	Description
<code>hotel</code>	Type of hotel (City Hotel or Resort Hotel).
<code>is_canceled</code>	Booking cancellation status (0 = Not Canceled, 1 = Canceled). Target.
<code>lead_time</code>	Number of days between booking date and arrival date.

arrival_date_year	Year of arrival.
arrival_date_month	Month of arrival.
arrival_date_week_number	Week number of the arrival date.
arrival_date_day_of_month	Day of the month of arrival.
stays_in_weekend_nights	Number of weekend nights (Sat/Sun) booked.
stays_in_week_nights	Number of weekday nights booked.
adults	Number of adults included in the booking.
children	Number of children included in the booking.
babies	Number of babies included in the booking.
meal	Type of meal package booked (e.g., BB, HB, FB, SC).
country	Country of origin of the guest.
market_segment	Market segment through which the booking was made (e.g., Online TA, Corporate, Direct).
distribution_channel	Booking distribution channel (e.g., Travel Agent, Direct, Corporate).
is_repeated_guest	Whether the customer is a returning guest (0/1).
previous_cancellations	Number of previous bookings canceled by the customer.
previous_bookings_not_canceled	Number of previous bookings completed successfully.
reserved_room_type	Room type originally reserved.
assigned_room_type	Room type assigned at check-in.
booking_changes	Number of modifications made before arrival.
deposit_type	Type of deposit associated with the booking.
agent	Unique identifier of the travel agent.
days_in_waiting_list	Number of days the booking stayed on the waiting list.
customer_type	Category of customer (e.g., Transient, Contract, Group).
adr	Average Daily Rate (average room price per night).
required_car_parking_spaces	Number of parking spaces requested.
total_of_special_requests	Total number of special requests made.
reservation_status	Final status of the reservation (Check-Out, Canceled, No-Show).
reservation_status_date	Date the reservation status was last updated.

3. Environment & Dependencies

- **Data access:** kagglehub
- **Core:** numpy, pandas, matplotlib, seaborn, tqdm
- **Modeling:** scikit-learn (multiple classifiers, Pipeline, ColumnTransformer, MinMaxScaler, OneHotEncoder, train_test_split, RandomizedSearchCV)
- **External boosting libraries:** xgboost, catboost (installed via %pip install catboost), lightgbm
- **Persistence:** joblib

4. Workflow

4.1 Data Loading

The raw CSV (`hotel_bookings.csv`) is downloaded via `kagglehub` and loaded into a pandas DataFrame.

4.2 Data Cleaning

1. Missing values inspected with `df.isna().sum()`.
2. The `company` column is **dropped** (very high missing-value rate).
3. Duplicate rows are removed (`drop_duplicates`) and the index is reset.
4. Remaining rows with missing `country` or `children` are dropped.
5. `is_canceled` is cast to `int`.

4.3 Exploratory Data Analysis (EDA)

- Boxplots for outlier inspection (overall and specifically `lead_time`).
- A correlation heatmap (`sns.heatmap`) over numeric features.
- Count plots comparing cancellation status against:
 - Hotel type (`hotel`)
 - Customer type (`customer_type`)
 - Market segment (`market_segment`)

4.4 Feature Selection (Leakage / Low-Value Column Removal)

The following columns are dropped before modeling to avoid data leakage or because they add little predictive value: - `agent` - `arrival_date_day_of_month` - `is_repeated_guest` - `previous_cancellations` - `reservation_status`, `reservation_status_date` (*direct leakage — these describe the outcome itself*)

4.5 Encoding

- Categorical columns (`select_dtypes(include='object')`) are transformed with `OneHotEncoder(sparse_output=False)`, output as a pandas DataFrame, and concatenated back with the numeric columns.

4.6 Train/Test Split (Baseline Models)

- 80/20 split via `train_test_split`, stratified on `is_canceled`, `random_state=100`.

5. Baseline Model Comparison

Twelve classifiers are trained and evaluated on the same train/test split:

Category	Models
Linear	Logistic Regression
Distance-based	K-Nearest Neighbors (KNN)
Tree	Decision Tree
Naive Bayes	Gaussian Naive Bayes
Bagging	Random Forest, Extra Trees
Boosting	AdaBoost, Gradient Boosting, Hist Gradient Boosting
External boosting libraries	XGBoost, CatBoost, LightGBM

(SVC is present in the code but commented out / excluded from the run.)

For each model, the notebook computes and reports: - Confusion matrix (visualized as a heatmap) - Full classification report (precision, recall, F1) - ROC-AUC and ROC curve - PR-AUC and Precision-Recall curve - Train/Test Accuracy, Precision, Recall, F1 Score, and Balanced Accuracy

A **soft-voting** `VotingClassifier` combining Logistic Regression, Random Forest, Naive Bayes, and XGBoost is also trained and evaluated alongside the individual models.

All results are collected into a `results_df` DataFrame, sorted by **Test F1 Score**, and visualized with bar charts comparing Accuracy, Precision, Recall, F1 Score, and Balanced Accuracy across models (train vs. test).

6. Production Pipeline (Deployment-Ready Model)

A separate, self-contained section rebuilds a leaner pipeline intended for deployment (e.g., a Gradio app):

1. **Reload & clean** the raw CSV using the same cleaning steps as the exploratory phase (drop `company`, remove duplicates, drop rows missing `country / children`, drop `reservation_status` and `reservation_status_date`).
2. **Reduce to 12 production features:**

Type	Features
Numeric (9)	<code>lead_time</code> , <code>stays_in_weekend_nights</code> , <code>stays_in_week_nights</code> , <code>adults</code> , <code>booking_changes</code> , <code>adr</code> , <code>total_of_special_requests</code> , <code>required_car_parking_spaces</code> , <code>previous_bookings_not_canceled</code>
Categorical (3)	<code>deposit_type</code> , <code>customer_type</code> , <code>market_segment</code>

3. **Build an sklearn Pipeline :**
 - `ColumnTransformer` with `MinMaxScaler` for numeric features and `OneHotEncoder(handle_unknown='ignore')` for categorical features.
 - Classifier: `XGBClassifier(eval_metric='logloss')`.
4. **Hyperparameter tuning** via `RandomizedSearchCV` :
 - Search space covers `n_estimators`, `max_depth`, `learning_rate`, `subsample`, `colsample_bytree`, `min_child_weight`, and `gamma`.
 - 60 iterations, 5-fold cross-validation, scored on accuracy.
5. **Evaluation** of the best estimator on the held-out test set (accuracy, classification report, confusion matrix).
6. **Persistence:** the entire fitted pipeline (preprocessing + model) is saved with `joblib.dump()` as `hotel_cancellation_pipeline.pkl`, so a downstream application (`app.py`) can load one file and make predictions directly on the 12 raw input features, without reimplementing any preprocessing logic.

7. Project Structure Summary

Stage	Purpose	Key Output
Setup & Imports	Load libraries, download dataset	—
Load Data	Read CSV into DataFrame	<code>df</code>
Data Cleaning	Handle missing values, duplicates	Cleaned <code>df</code>
EDA	Understand distributions & correlations	Plots
Feature Selection	Remove leakage/low-value columns	Reduced <code>df</code>
Encoding	One-hot encode categoricals	Numeric-only <code>df</code>
Baseline Modeling	Compare 12 classifiers + voting ensemble	<code>results_df</code>
Production Pipeline	Rebuild with 12 features, tune XGBoost	<code>best_pipeline</code>
Deployment Prep	Serialize final pipeline	<code>hotel_cancellation_pipeline.pkl</code>

8. Notes

- The confusion matrix labels in the baseline loop ('Not Stroke' / 'Stroke') appear to be leftover placeholder labels from a different project template and do not reflect the actual classes (Not Canceled / Canceled) — worth correcting for clarity if this notebook is reused.
- The final production pipeline intentionally uses a smaller, curated feature set (12 features) rather than the full one-hot-encoded feature space used in the baseline comparison, to keep the deployed app's input form simple.